API WebSocket

Comunicación bidireccional y en tiempo real sobre una sola conexión

Integrantes

Juan Diego Grajales
Juan Pablo Sarmiento
Jan Sánchez



Estilos arquitectónicos de API · 2026

Agenda: 20 minutos en 8 bloques



Juan Diego (6 min)

1. Contexto
2. Cómo funciona

Juan Pablo (6 min)

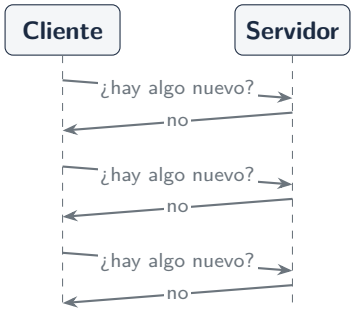
3. Vocabulario clave
4. Ventajas y desventajas
5. Cuándo usarlo

Jan (7 min)

6. Comparación con REST
7. Demostración en vivo
8. Cierre (todos, 1 min)

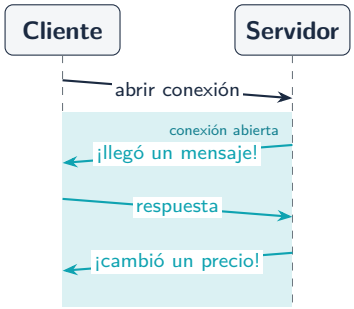
El problema: en HTTP el servidor no puede hablar primero

Antes: sondeo (*polling*)



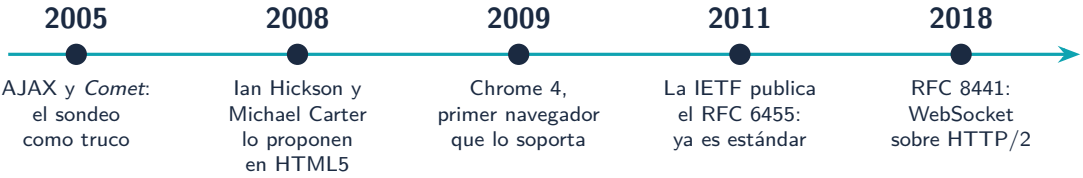
Desperdicio: la mayoría de respuestas son "no", y cada pregunta repite todas las cabeceras HTTP.

Con WebSocket



Solución: una conexión abierta por la que el servidor avisa en cuanto algo cambia.

De un truco a un estándar: 2005 a 2018



¿Quién lo impulsó? Los equipos de navegadores (WHATWG) y luego la IETF.

¿Por qué así? Empieza como una petición HTTP para usar los mismos puertos (80 y 443) y pasar por la infraestructura web existente.

En una frase: WebSocket nació para que el navegador y el servidor puedan hablar en cualquier momento, sin trucos.

WebSocket en tres preguntas

¿Cómo se comunican?

Bidireccional simultáneo (*full-duplex*).

Cualquiera de los dos envía cuando quiere. No hay petición y respuesta.

¿Sobre qué protocolo?

Protocolo propio (**RFC 6455**) sobre **TCP**.

`ws://` puerto 80
`wss://` puerto 443, cifrado con TLS

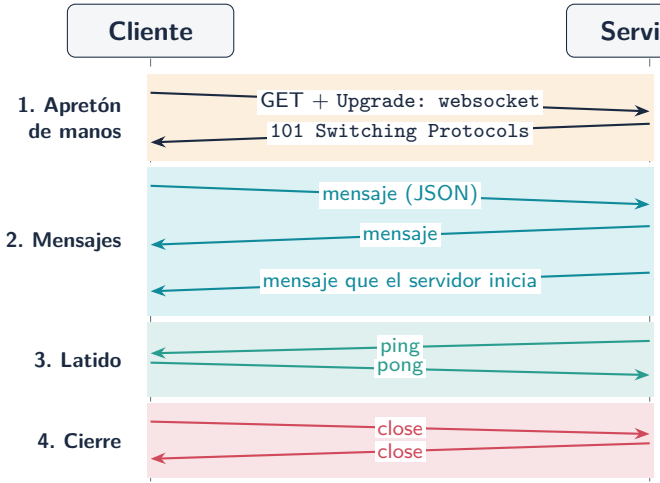
¿En qué formato?

Mensajes de **texto** (UTF-8, casi siempre JSON) o **binarios**.

La aplicación decide qué significa cada mensaje.

En una frase: una conexión TCP que se queda abierta y por la que viajan mensajes en ambos sentidos.

Ciclo de vida de una conexión (diagrama propio)



Fase 1: el apretón de manos convierte HTTP en WebSocket

El cliente pide cambiar de protocolo

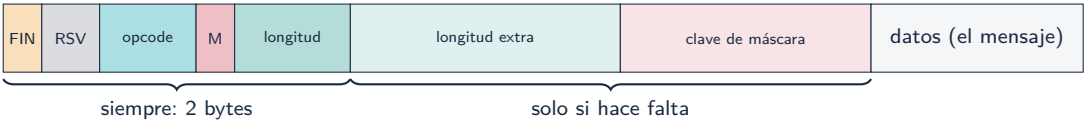
```
GET /chat HTTP/1.1
Host: localhost:8080
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
```

El servidor acepta

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+
    xOo=
```

- `Upgrade`: "quiero dejar de hablar HTTP".
- **101**: "aceptado". Desde aquí ya no es HTTP.
- `Sec-WebSocket-Accept`: la clave del cliente procesada con SHA-1. Prueba que el servidor de verdad entiende WebSocket.
- La conexión TCP **no se cierra**: se reutiliza para todo lo demás.

Fase 2: cada mensaje viaja en tramas (*frames*) muy livianas



El *opcode* dice qué tipo de trama es

`0x1`	texto
`0x2`	binario
`0x8`	cierre
`0x9` / `0xA`	ping / pong

- **FIN**: marca la última trama de un mensaje grande.
- **M** (máscara): lo que envía el cliente va enmascarado para no confundir a los proxies.

En una frase: la cabecera ocupa entre 2 y 14 bytes; una petición HTTP suele cargar cientos.

Vocabulario clave: seis términos que ya vimos

Handshake · *apretón de manos*

Petición HTTP inicial con la que ambos acuerdan pasar a WebSocket.

Full-duplex · *bidireccional simultáneo*

Los dos extremos envían al mismo tiempo, como en una llamada telefónica.

Frame · *trama*

Unidad mínima que viaja por la conexión. Una o varias forman un mensaje.

Opcode · *código de operación*

Indica si la trama es texto, binario, cierre o latido.

Ping / Pong · *latido*

Tramas de control para saber si la otra parte sigue conectada.

Subprotocol · *subprotocolo*

Acuerdo sobre el formato de los mensajes (por ejemplo, STOMP), negociado en el apretón de manos.

Ventajas y desventajas

Ventajas

1. Baja latencia
Ej.: un precio de bolsa se actualiza en milisegundos.

2. El servidor puede hablar primero
Ej.: aviso de "nuevo mensaje" sin recargar la página.

3. Poca sobrecarga por mensaje
Ej.: un juego que envía 30 posiciones por segundo.

4. Soporte nativo en navegadores
Ej.: basta `new WebSocket(url)`, sin librerías.

Desventajas

1. Conexiones con estado: difícil de escalar
Ej.: A en el servidor 1 no ve a B en el servidor 2 sin un bus como Redis.

2. Sin reconexión ni entrega garantizada
Ej.: un túnel del metro corta el chat y hay que reconectar a mano.

3. Sin caché ni reglas estándar
Ej.: no hay verbos como GET o POST; el formato lo inventa uno.

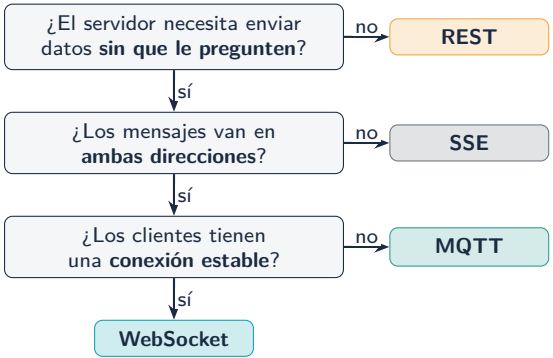
4. Proxies y cortafuegos
Ej.: redes corporativas que cortan conexiones largas.

¿Quién lo usa y para qué?

Empresa	Qué viaja por WebSocket
Discord	Su *Gateway* entrega mensajes, presencia (“en línea”) y eventos de los servidores a cada cliente y bot.
Slack	Con *Socket Mode*, las apps reciben eventos sin exponer una URL pública.
Binance	Flujos de precios, órdenes y operaciones del mercado en tiempo real.
Figma	Los cambios de cada persona en un diseño compartido, al instante.

En una frase: todos tienen muchos eventos pequeños y frecuentes, donde un segundo de retraso arruina la experiencia.

¿Cuándo no usarlo?



Mal caso: catálogo de productos

Los usuarios consultan de vez en cuando y los datos casi no cambian.

Con WebSocket se paga una conexión abierta por usuario **sin ningún beneficio**, y se pierden la caché y la simplicidad de REST.

SSE: el servidor envía en un solo sentido.

MQTT: mensajería para dispositivos con red inestable.

WebSocket frente a REST

	WebSocket	REST
Modelo	Mensajes en ambas direcciones	Petición y respuesta
Quién inicia	Cliente o servidor	Siempre el cliente
Conexión	Una, persistente y con estado	Una por petición, sin estado
Sobrecarga	2 a 14 bytes por mensaje	Cabeceras HTTP completas
Caché	No existe	Nativa de HTTP y de las CDN
Contrato	Libre; se describe con AsyncAPI	Verbos, URLs y OpenAPI
Escalar	Difícil: conexiones abiertas	Fácil: balanceo simple

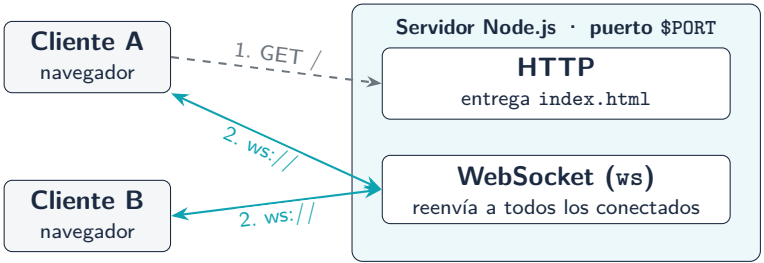
Se gana

Tiempo real, envío desde el servidor y menos tráfico.

Se pierde

Simplicidad, caché, herramientas estándar y escalado fácil.

Nuestra demo: un mini chat (diagrama propio)



1. El navegador descarga la página por HTTP.

2. Abre un WebSocket **al mismo puerto**.

3. Lo que escribe A le llega a B al instante.

En una frase: cliente y servidor en procesos separados, con una única conexión abierta por navegador.

Demostración en vivo: qué vamos a ver

	Qué hacemos	Qué demuestra
1	Arrancar el servidor: `npm start`	Servidor y clientes son procesos separados
2	Abrir dos navegadores	Ambos reciben "Hola mundo!": **el servidor habla primero**
3	Escribir en A	Aparece al instante en B: **bidireccional, sin recargar**
4	*DevTools* → Red → WS	Una sola petición **101** y tramas subiendo y bajando: **no hay sondeo**
5	Apagar el servidor	Los clientes reciben `close`: la conexión tiene estado

El código esencial

server.js (proceso 1)

```
import { WebSocketServer, WebSocket } from 'ws';
import 'dotenv/config';
import http from 'node:http';
const PORT = Number(process.env.PORT);
const servidorHttp = http.createServer(/* ... */);
const wss = new WebSocketServer(
  { server: servidorHttp }); // mismo puerto
wss.on('connection', (socket) => {
  socket.send(JSON.stringify(
    { autor: 'servidor', texto: 'Hola mundo!' }));
  socket.on('message', (data) => {
    for (const c of wss.clients) { // broadcast
      if (c.readyState === WebSocket.OPEN)
        c.send(data.toString());
    }
  });
});
servidorHttp.listen(PORT);
```

index.html (procesos 2 y 3)

```
const ws = new WebSocket(WS_URL);
ws.onmessage = (e) => mostrar(JSON.
    parse(e.data));
boton.onclick = () =>
  ws.send(JSON.stringify({ autor,
    texto }));
```

.env.example

```
PORT=8080
WS_URL=ws://localhost:8080
```

- on('connection'): alguien abrió un WebSocket.
- on('message'): llegó una trama; se reenvía a todos.
- onmessage: el navegador muestra lo que llega.

Tres ideas para llevarse

1

Es **una conexión persistente y bidireccional** que nace como HTTP y luego cambia de protocolo.

2

Brilla cuando el **servidor necesita hablar primero** y con frecuencia: chats, juegos, mercados.

3

La complejidad pasa a nosotros: **reconexión, escalado y formato** de los mensajes.

Fuentes consultadas

Especificaciones

- IETF. *RFC 6455: The WebSocket Protocol* (2011). `https://datatracker.ietf.org/doc/html/rfc6455`
- IETF. *RFC 8441: Bootstrapping WebSockets with HTTP/2* (2018). `https://datatracker.ietf.org/doc/html/rfc8441`
- WHATWG. *WebSockets Standard*. `https://websockets.spec.whatwg.org`

Documentación técnica

- MDN Web Docs. *API de WebSockets*. `https://developer.mozilla.org/es/docs/Web/API/WebSockets_API`
- Librería `ws` para Node.js. `https://github.com/websockets/ws`

Casos de uso

- Discord Developer Docs. *Gateway*. `https://discord.com/developers/docs/events/gateway`
- Slack API. *Socket Mode*. `https://api.slack.com/apis/socket-mode`
- Binance. *WebSocket Streams*. `https://developers.binance.com/docs/binance-spot-api-docs/web-socket-streams`
- Figma Blog. *How Figma's multiplayer technology works*. `https://www.figma.com/blog/how-figmas-multiplayer-technology-works/`

¿Preguntas?

Juan Diego Grajales · Juan Pablo Sarmiento · Jan Sánchez